MultiSportPredict
Czech Liga Pro Match & Command Guide

Updated 10 October 2026 • Windows / Git Bash / VS Code terminal

WHERE TO PASTE: commands go into Git Bash, including a VS Code terminal set to Git Bash. Notepad is only for copied sportsbook page text. Do not paste commands into Notepad.

HOW TO USE: copy only the command lines. Replace every bold value with your own information before pressing Enter. Keep the quotation marks. Do not type Markdown asterisks. Commands without bold values can be pasted exactly as shown.

DATA INGESTION: QUICK START

FREE DATA COLLECTION — paste the following command lines into Git Bash. They collect finished Czech Liga Pro matches and attempt training. No paid token is needed.

COPY INTO GIT BASH:

```
cd /c/MultiSportPredict

source venv/Scripts/activate

venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 10 --auto-train

venv/Scripts/python.exe -u ingest_table_tennis.py --stats
```

This merges verified final scores, exports history and attempts training. It does not fetch MyStake prices; use the copied-page match workflow below for current prices. A small history is collected data, not a trained recommendation.

CONTINUOUS COLLECTION — run in a separate Git Bash window and leave it open. Press Ctrl+C to stop. The delay after each cycle is 30 minutes.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100 --watch --interval 1800 --auto-train
```

Export the existing store to CSV without collecting another page:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --export-only
```

DISCORD AFTER A MATCH — ingestion does not create a prediction card. To send your latest saved match report, paste this command into Git Bash:

```
venv/Scripts/python.exe -u run_table_tennis.py --push latest
```

All other sports:
https://docs.google.com/document/d/1W21N_tqpREVkbQhdGND1uclTc7YvfvsYQRLcC__ZopI/edit

Action index

- DATA INGESTION: QUICK START
- Start Git Bash or VS Code
- Run an upcoming match: complete free workflow
- Run a second match or refresh prices
- View saved results in your terminal
- Preview and push to Discord
- Read probabilities, prices and EV
- DATA INGESTION: collect and refresh table-tennis history
- Train and inspect player history
- Grade predictions after matches finish
- Manually update a verified MyStake final result
- Import a verified JSON export or source snapshot
- Check win percentage versus point-cover percentage
- API INGESTION: setup, backfill and fixture refresh
- Troubleshooting and dependency repair
- File locations and command lookup
- Ordinary tennis: ATP / WTA appendix

Start Git Bash or VS Code

STEP 1 — Open Git Bash from Windows Start. Alternatively, in VS Code choose Terminal → New Terminal and select the Git Bash terminal profile. Paste the following setup commands into that terminal and press Enter.

COPY INTO GIT BASH:

```
cd /c/MultiSportPredict

source venv/Scripts/activate

pwd

venv/Scripts/python.exe --version
```

Expected: /c/MultiSportPredict, a Python version and usually (venv). All commands use the explicit project interpreter. -u displays output immediately. If a collector occupies this terminal, press Ctrl+C once and wait for the $ prompt before running a match, or use a separate Git Bash terminal for the match.

Run an upcoming match: complete free workflow

Copy and save the sportsbook text

STEP 2 — In your browser, open one upcoming MyStake Czech Liga Pro match. Expand the markets you want. Copy the visible text containing the league, both player names, date/time, market labels and both prices. Check that the copied text actually contains those items. A screenshot cannot be used as page text.

STEP 3 — Paste the next command into Git Bash. It opens the exact text file in Notepad.

COPY INTO GIT BASH:

```
notepad.exe 'C:\MultiSportPredict\data\table_tennis\current_match.txt'
```

NOTEPAD ONLY — Paste the copied sportsbook page text into the Notepad window. Press Ctrl+S to save, then close Notepad. Replace this file with one complete match page at a time. Do not paste Python commands into this file or invent a missing league heading.

Run the match and print the report

STEP 4 — Return to Git Bash. Replace the bold values in the command below, paste the complete command into Git Bash, and press Enter. Run within five minutes of copying the page and before the match starts.

REPLACE: First Player vs Second Player = both actual names; MyStake Match URL = the individual prematch page URL; YYYY-MM-DDTHH:MM:SS-TIMEZONE_OFFSET = the verified match start with its actual UTC offset.

TIME FORMAT: use a date, T, 24-hour time and numeric offset. Eastern daylight time uses -04:00; Eastern standard time uses -05:00. Use the offset for the fixture date. Do not enter EST or TIMEZONE_OFFSET literally.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --match "First Player vs Second Player" --mystake-url "MyStake Match URL" --mystake-text-file data/table_tennis/current_match.txt --start-utc "YYYY-MM-DDTHH:MM:SS-TIMEZONE_OFFSET" --best-of 5
```

The saved page file supplies the observed markets and prices. --best-of 5 confirms the actual best-of-five format. A successful run prints the predictions and a Saved Run ID starting with tt_. Save that ID to review or send this exact report later.

Optional automatic browser attempt

This attempts to read the page without the copied file. MyStake browser capture has been unreliable in this project; use the copied-page workflow if it fails.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --match "First Player vs Second Player" --mystake-url "MyStake Match URL" --start-utc "YYYY-MM-DDTHH:MM:SS-TIMEZONE_OFFSET" --best-of 5
```

Run a second match or refresh prices

For another match: copy its complete page into the same Notepad file, save, then return to Git Bash and run the match command with the new bold values. Use one match page per file. To refresh prices, recopy the current page before rerunning. Merely resaving old text does not refresh prices. This workflow is for upcoming prematch fixtures.

View saved results in your terminal

SHOW LATEST — display your most recent saved prediction. Paste this command into Git Bash:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --show latest
```

```
venv/Scripts/python.exe -u run_table_tennis.py --list-runs
```

SHOW A SPECIFIC REPORT — replace Saved Run ID with the tt_ identifier printed by that run. Paste the command into Git Bash:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --show "Saved Run ID"
```

EXISTING SAVED REPORTS — these commands display previously captured output; they do not run a new match or fetch current odds.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --show tt_d607a0e572314dc89db15adc0189a350
```

```
venv/Scripts/python.exe -u run_table_tennis.py --show tt_ae4708ed150641bbab07de865a265979
```

The first is Vaclavik–Havrda; the second is Holovatiuk–Briza. --show reads a saved report without collecting prices or creating another forecast.

Preview and push to Discord

PREVIEW A SPECIFIC REPORT — replace Saved Run ID with the saved tt_ identifier. This previews Discord content without sending it.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --push "Saved Run ID" --dry-run
```

The dry run previews the payload without sending. To send the report:

SEND A SPECIFIC REPORT — replace Saved Run ID with the same saved tt_ identifier. Paste the command into Git Bash to send it.

Push uses configured Discord destinations and the latest formatter with the report's original prices. It may update the prior tracked card for that fixture. Model Notes is removed from Discord; the research/price basis remains in the short summary. Collection, training and grading do not generate new prediction cards. After those actions, the push command above sends an associated saved prediction; it does not create a grading-results card.

Read probabilities, prices and EV

Priced markets: the copied page or API supplies actual paired sportsbook odds. The report shows decimal odds, model win probability and EV. EV = P(win) × (decimal odds − 1) − P(loss). Pushes return the stake and contribute zero. +5% EV means a calculated 0.05 units of expected profit per unit staked under the model, not a guarantee.
Model references: the model generates probabilities at reference lines. A sportsbook quote may be absent for those lines. Without actual odds there is no sportsbook price or EV to report. A quote for Over 73.5 cannot price Over 77.5. If your copied page supplies a recognized market and both actual prices, it appears in the priced section. Missing or ambiguous lines remain projections.
Supported market families: match winner; match total games; game handicap; match total points; point handicap; first game winner; first game total points; first game point handicap. Actual availability depends on the source. In handicaps, A is the first player displayed by the source; the displayed B handicap has the opposite sign. Current market-anchored output does not provide an independently trained betting edge.

DATA INGESTION: collect and refresh table-tennis history

Collect visible recent finished Czech Liga Pro results once, export CSV and attempt training:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 10 --auto-train
```

Keep collecting with a 30-minute delay after each cycle:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100 --watch --interval 1800 --auto-train
```

Leave that terminal open; Ctrl+C stops. Discovery, each accepted/rejected match and the waiting interval are printed. Recent results are limited, not a complete backfill. Access can fail; only complete verified legal scores are saved. Duplicate results do not create extra events.

Check store counts, export CSV or inspect files:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --stats
```

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --export-only
```

```
explorer.exe 'C:\MultiSportPredict\data\table_tennis'
```

Collect one known completed public match:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --match-url "Completed Sofascore Match URL" --auto-train
```

Sofascore history uses different player/event IDs from MyStake. Automatic reconciliation is still unresolved. Collecting a result under a Sofascore ID alone does not grade a MyStake prediction.

Train and inspect player history

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u train_table_tennis.py
```

```
venv/Scripts/python.exe -u train_table_tennis.py --export data/table_tennis/training_rows.json
```

```
venv/Scripts/python.exe -u train_table_tennis.py --diagnostic
```

The commands respectively fit the model, export eligible feature rows, and run the losing-point Poisson diagnostic. Training requires at least 150 eligible matches; each eligible match needs eight prior matches for each player. An insufficient-data message means no model was fitted. Completed results alone are not equivalent to a validated betting model. The diagnostic is not a separate recommendation.

Grade predictions after matches finish

Refresh completed history using your chosen source, then grade:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100
```

```
venv/Scripts/python.exe -u grade_table_tennis.py
```

Grading needs the saved report's exact event/player identities, finished status, and all legal per-game point scores ending at three game wins. A final 3–1 match score alone is insufficient. For MyStake reports, use the verified manual-result update below when automatic identity reconciliation is unavailable.

Output reports graded selections, unfinished/invalid runs, hypothetical one-unit profit and prospective probability metrics where available. These are research outcomes; the engine did not record that you placed those bets. Regrading updates records for corrected results. Discord push continues to send the saved prediction, not this grading summary.

Manually update a verified MyStake final result

ADVANCED: VERIFIED FINAL RESULT — use only after confirming the completed match, date/time, both players and every game score. Commands below go into Git Bash. Replace Saved Run ID with the saved tt_ identifier. Replace the bold score array with the actual per-game point pairs in source A/B order. Include every completed game; a match score such as 3–1 is insufficient. Replace Verified Final Result URL with the page where you verified those scores.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --show "Saved Run ID"
```

BUILD THE VERIFIED IMPORT FILE — replace all bold values, then paste this complete single command into Git Bash. Its Python parentheses are executable code and must be copied. This preserves stored identities after your manual confirmation; it does not automatically verify the result page.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -c 'import sys,json; from pathlib import Path; from
core.historical_storage import TableTennisStore; from table_tennis.contracts
import utcnow,validate_event; r=TableTennisStore().run(sys.argv[1]);
e=dict(r["event"]);
e.update(status="finished",game_scores=json.loads(sys.argv[2]),current_points=[
0,0],retrieved_utc=utcnow(),source_url=sys.argv[3]); e.pop("end_utc",None);
e=validate_event(e); p=Path("data/table_tennis/verified_final_result.json");
p.write_text(json.dumps({"events":[e],"quotes":[]},indent=2),encoding="utf-8");
print("Validated result written:",p)' "Saved Run ID"
"[[A_POINTS,B_POINTS],[A_POINTS,B_POINTS],[A_POINTS,B_POINTS]]" "Verified
Final Result URL"
```

Only after that command succeeds, import and grade:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --import-file
data/table_tennis/verified_final_result.json

venv/Scripts/python.exe -u grade_table_tennis.py

venv/Scripts/python.exe -u train_table_tennis.py
```

If validation fails, correct the actual input before importing. Unknown or incomplete final points must remain ungraded. This manually confirmed update does not claim that the engine independently verified the result-page identity. Importing a corrected result uses the same workflow and retains the database's event versions.

Import a verified JSON export or source snapshot

A normalized engine export has events/quotes arrays. Raw DOM snapshots, page-text files and CSV exports use different formats.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --import-file "Normalized JSON File Path"
```

A raw Sofascore DOM capture produced by the public collector uses this importer:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --snapshot-file "Public Snapshot JSON File Path"
```

After normalized results are imported, use the grading/training commands above. Exported results.csv is an inspection file, not a normalized JSON importer input.

Check win percentage versus point-cover percentage

Compare wins and point covers for one player using the same historical bookmaker quotes. Replace Stored Player ID with an exact ID from results.csv or saved JSON, Signed Point Handicap with the real signed number, and Bookmaker with its stored name. Matching quotes and settled results are required.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --win-cover-player-id "Stored Player ID" --point-handicap "Signed Point Handicap" --bookmaker "Bookmaker"
```

Player IDs appear in results.csv and saved report JSON. Insufficient matching samples remain unavailable. This comparison is descriptive, not EV.

API INGESTION: setup, backfill and fixture refresh

Optional: obtain a separate BetsAPI token with verified Czech Liga Pro completed game-point coverage and the required odds entitlement. Your existing The Odds API key has no table-tennis coverage, and a Bet365 login is not an API token. Enter tokens privately in your own terminal.

COPY INTO GIT BASH:

```
winpty venv/Scripts/python.exe configure_table_tennis.py

venv/Scripts/python.exe -u configure_table_tennis.py --check

venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 90 --no-quotes --auto-train
```

Refresh current fixtures and available sportsbook quotes (requires a BetsAPI token and odds coverage):

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 0
```

PAID PROVIDER MATCH RUN — requires configured BetsAPI access. Replace both bold player names, then paste the command into Git Bash.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_table_tennis.py --provider betsapi --match "First Player vs Second Player"
```

Refresh recent provider results and grade provider-matched runs:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 1 --no-quotes --auto-train

venv/Scripts/python.exe -u grade_table_tennis.py
```

SEND THE ASSOCIATED SAVED PREDICTION — this sends a match prediction, not a grading summary. Replace Saved Run ID and paste into Git Bash:

```
venv/Scripts/python.exe -u run_table_tennis.py --push "Saved Run ID"
```

Provider history does not automatically reconcile earlier MyStake identities.

Troubleshooting and dependency repair

No $ prompt: an action still occupies the terminal. Watch status says when it is waiting; Ctrl+C stops it. A lone > prompt means an unfinished quote/multiline command; Ctrl+C, then paste a complete command. File not found: check pwd and cd /c/MultiSportPredict.
Copied page rejected: ensure the actual page contains all league labels, both players and paired market prices. Recopy current prices within five minutes. Match start passed: use --show for the saved report; do not change the start to a fake future time. Browser source blocked: use fresh copied text for MyStake; public results retain old data on failure.
No EV: actual prices are absent for that market/line. No trained model: collect eligible history and run training. Zero grades: final scores or exact event identities are missing. CSV access denied: close results.csv in Excel before another export.
Dependency repair only when the error says dependencies/browser are missing:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -m pip install -r requirements-table-tennis.txt

venv/Scripts/python.exe -m playwright install chromium
```

File locations and command lookup

Project: C:\MultiSportPredict
Copied match: data\table_tennis\current_match.txt
Canonical history: multisport_history.db
CSV export: data\table_tennis\results.csv
Raw completed captures: data\table_tennis\public_raw
Fitted model: data\table_tennis\model.json (only after training succeeds)
Saved reports: output\table_tennis\tt_....json and tt_....md
Command guide local copy: TABLE_TENNIS_COMMANDS.md

COPY INTO GIT BASH:

```
venv/Scripts/python.exe run_table_tennis.py --help
```

```
venv/Scripts/python.exe ingest_table_tennis_public.py --help
```

```
venv/Scripts/python.exe ingest_table_tennis.py --help
```

```
venv/Scripts/python.exe train_table_tennis.py --help
```

```
venv/Scripts/python.exe grade_table_tennis.py --help
```

Ordinary tennis: ATP / WTA appendix

Use run_tennis.py or run_tennis_full.py for ordinary tennis. These have different storage/review conventions and Discord flags from run_table_tennis.py.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u ingest_tennis.py
```

Standard match run: enter the actual fixture properties rather than accepting a default tournament.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_tennis.py --match "First Player vs Second Player" --surface "Surface" --tournament "Tournament" --round "Round" --best-of "Best Of" --game-date "YYYY-MM-DD"
```

Push a selected review row from the standard runner:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_tennis.py --push "Review Row Number"
```

The full research-package runner fetches a source fixture/profile/odds package where supported; source access and identifiable history remain necessary:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u run_tennis_full.py --match "First Player vs Second Player" --refresh
```

```
venv/Scripts/python.exe -u run_tennis_full.py --push
```

Grade logged ordinary-tennis predictions with the ordinary project grader. The full research-package runner saves a research report and does not automatically register a graded bet.

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u grade_predictions.py --pending --sport tennis
```

```
venv/Scripts/python.exe -u grade_predictions.py --auto --sport tennis --report
```

Push the ordinary-tennis grading record to Discord:

COPY INTO GIT BASH:

```
venv/Scripts/python.exe -u grade_predictions.py --report --sport tennis
--push-discord
```